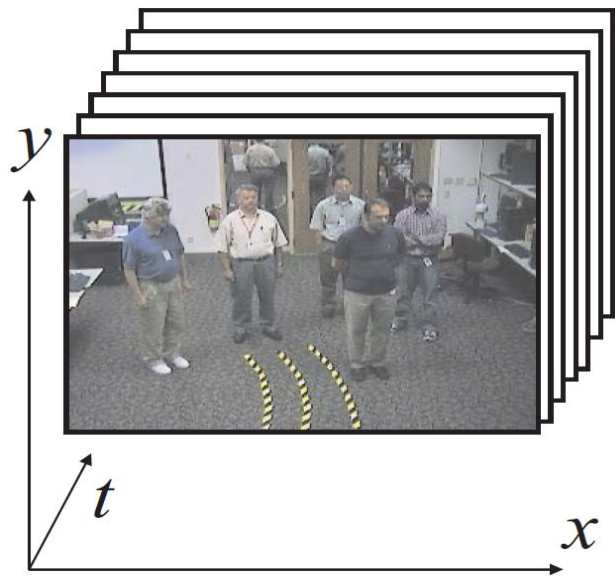


Computer vision

Motion detection

Vanda Benešová

Dynamic scene analysis



The input to the dynamic scene analysis is a sequence of image frames $F(x, y, t)$ taken from the changing world.

x, y are spatial coordinates.
Frames are usually captured at fixed time intervals.

t represents $t_{\text{-th}}$ frame in the sequence.

Typical Applications Areas

Motion detection. Often from a static camera. Common in surveillance systems. Often performed on the pixel level only (due to speed constraints).

Motion segmentation. Images are segmented into region corresponding to different moving objects.

Object tracking. A sparse set of features is often tracked, e.g., corner points.

Three-dimensional shape from motion. Called also structure from motion (SFM). Similar problems as in stereo vision.

Typical Applications

automated surveillance: monitoring a scene to detect suspicious activities or unlikely events

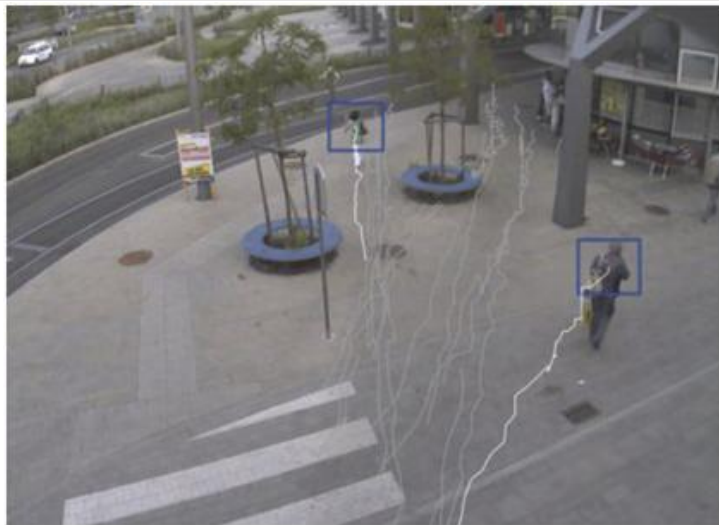
video indexing: automatic annotation and retrieval of videos in multimedia databases

human-computer interaction: gesture recognition, eye gaze tracking for data input to computers, etc.

traffic monitoring: real-time gathering of traffic statistics to direct traffic flow

vehicle navigation: video-based path planning and obstacle avoidance capabilities

Examples: Pedestrians



Improved pedestrian tracking for urban planning
[Oliver Sidla]



Crowded Scene in an Underground Station.
Pedestrians who cross the bright (blue) line at
the foot of the elevator are being counted.

Background Modeling

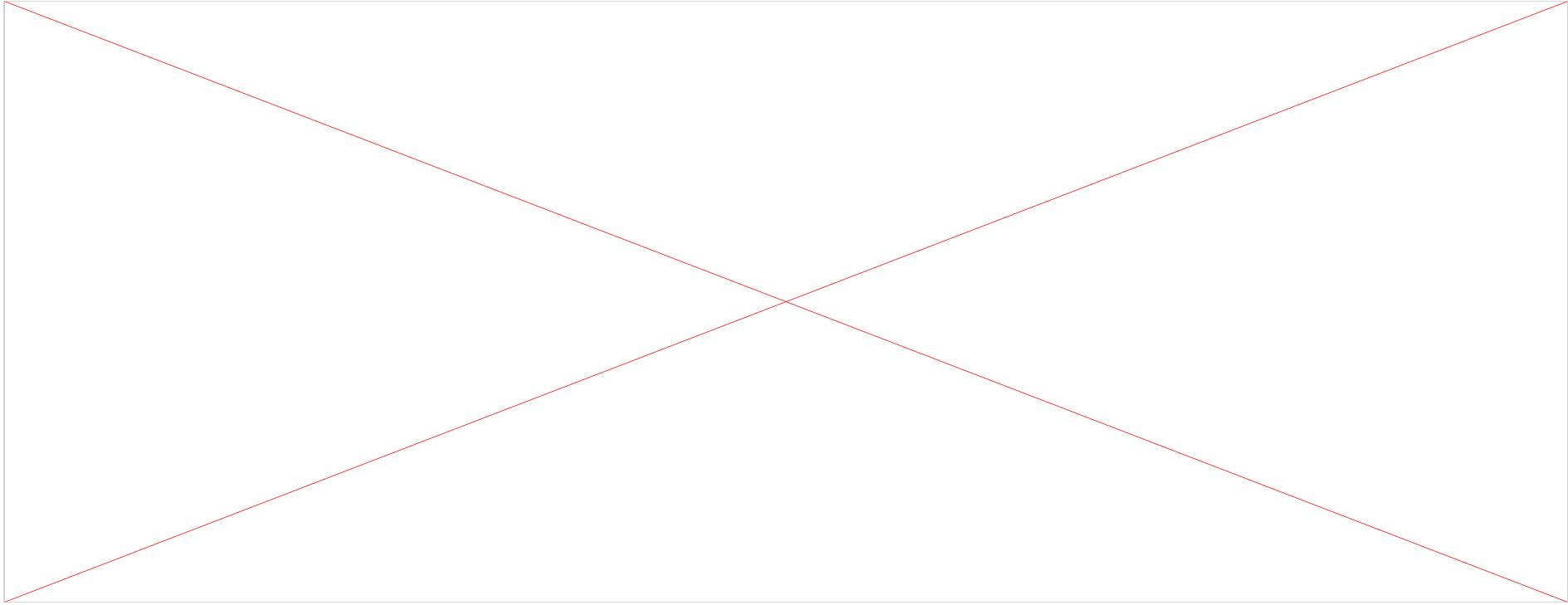
! stationary position of the sensor (camera)!

Approaches of Background Modeling

- Background Subtraction
- Statistical Methods
(e.g., Gaussian Mixture Model, Stauffer and Grimson 2000)

Background Subtraction

Differential motion analysis methods



Differential motion analysis methods -example

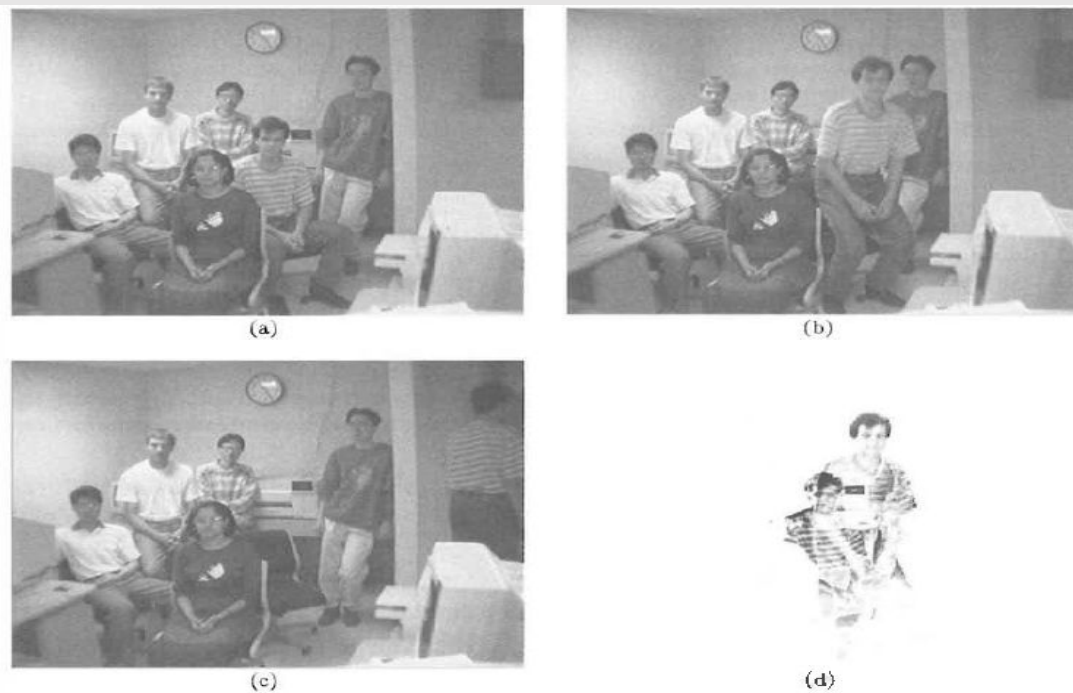


Figure 16.2: Motion detection. (a) First frame of the image sequence. (b) Frame 2 of the sequence. (c) Last frame (frame 5). (d) Differential motion image constructed from image frames 1 and 2 (inverted to improve visualization).

Background Subtraction

Background model computed as an **average or a median** of the previous n frames - **running average (median)**

1. Construct a background image B as a updating weighted average (or median) of n last images

Background Subtraction:

1. For each actual frame F , classify individual pixels as foreground if $|B-F| > T$ (*threshold*)
2. Clean noisy pixels

Background modelling - Running average

Running average:

$$B_{i+1} = \alpha * F_i + (1 - \alpha) * B_i$$

α - the learning rate (typically 0.05)

OpenCv

C++

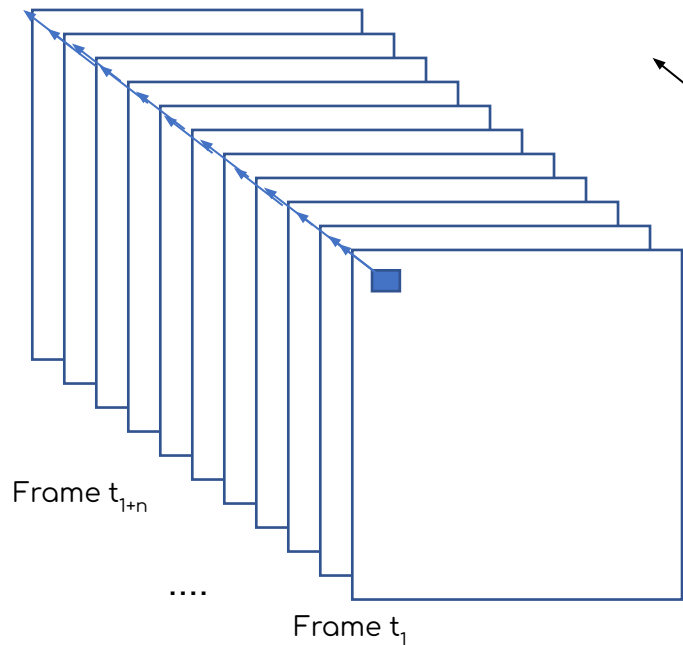
```
void cv::addWeighted (InputArray src1, double alpha, InputArray src2, double beta, double gamma, OutputArray dst, int dtype = -1)
```

Python:

```
cv.addWeighted( src1, alpha, src2, beta, gamma[, dst[, dtype]] ) -> dst
```

Statistical Method - Gaussian Mixture Model

Mixtures of Gaussians (MOG)

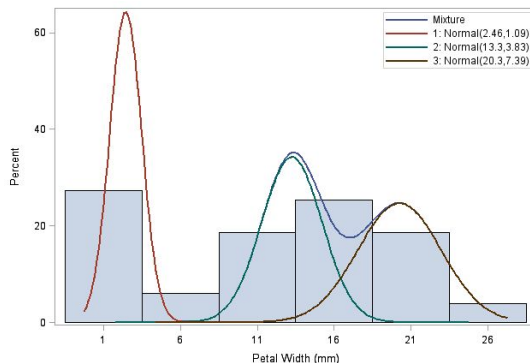
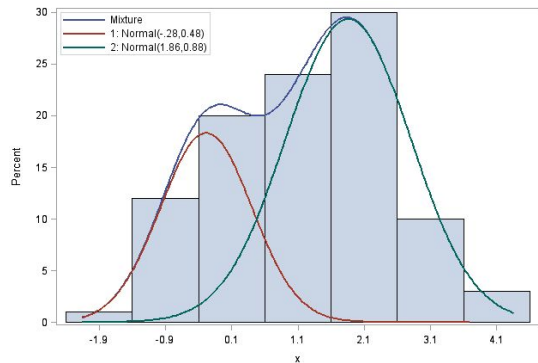


GMM model based on time evaluation of each pixel

For each pixel -> adaptive MOG model

Mixtures of Gaussians (MOG)

A linear superposition of Gaussian components



MOG2 - selects the appropriate number of gaussian distribution for each pixel.

Values of each pixel (in the time) are modeled with a mixture of Gaussians.

Number of Gaussians is varying.

Default number in MOG2 in OpenCv =5

The mixture of Gaussians actually models both :

the foreground and the background

Mixtures of Gaussians (MOG)

All observations **at each pixel** position $[x, y]$ are modeled with a mixture of Gaussians to estimate the background.

The probability distribution density of the pixel values can now be described by

$$\sum_i w_i \cdot N_{\underline{\xi}}(\underline{\mu}_i, \Sigma_i)$$

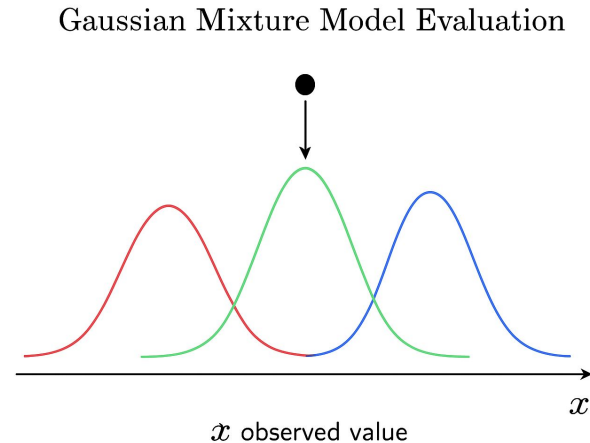
$N_{\underline{\xi}}$ is the multivariate Gaussian with mean $\underline{\mu}_i$
and covariance matrix Σ_i .

w_i are mixing coefficients - weights (sum of $w_i = 1$)

MOG - Update of the model for a new observation

When a **new observation x** arrives, it is checked if it is similar to already modeled observations or if it is originating from a new object. (It may also be just the noise.)

-> it will be evaluated **which of the associated Gaussians is the nearest.**



MOG Determining the corresponding Gaussian & Update

For each new observation x , it is checked whether it resembles an observation already modeled with MOG or originates from a new object. (It may also be just the noise.)

This is done by evaluating the **Mahalanobis distance** between the observation x and the i -th Gaussian modeled by the μ_i and σ_i .

Mahalanobis distance:

$$\delta(\underline{x}, \underline{\mu}_i) = \sqrt{(\underline{\mu}_i - \underline{x})^T \Sigma_i^{-1} (\underline{\mu}_i - \underline{x})}$$

where: x is the pixel value ,

μ_i is mean value ,

Σ_i is covariance matrix

MOG Determining the corresponding Gaussian & Update

If for next observations x is true that:

Mahalanobis distance from the i -th Gaussian $\delta(x, \mu_i) < T_{near}$

where T_{near} is a given constant (for example: $T_{near} = 2.5 \sigma_i$)

then $\rightarrow$ the “membership” to this corresponding i -th Gaussian will be confirmed and the model of the i -th Gaussian given by μ_i and σ_i will be updated by the observation x .

This implies that μ_i and σ_i of the i -th Gaussian are updated using a running average.

Otherwise $\rightarrow$ a new Gaussian will be created and added to the mixture.

Background Model Estimation

For each pixel, MOG2 maintains several Gaussians and we need to determine **which of the Gaussians of the mixture are most likely produced by background** processes.

For each Gaussian i , its **weight w_i** reflects:
how frequently that Gaussian has matched the pixel recently

$$0 \leq w_i \leq 1, \quad \sum w_i = 1$$

All weights w_i are updated (updated and/or normalized) at every new frame for every pixel

Background Model Estimation

- all distributions are ranked according to their w_i/σ_i
- largest w_i/σ_i will be chosen as “background”

Intuition:

A high weight w_i means the Gaussian often matches the pixel and low standard deviation σ_i means the gaussian is more “stable” → likely background.

A low weight means the Gaussian rarely matches and high standard deviation σ_i means the gaussian is less “stable” → likely foreground / transient object.

Thus, weight w_i is key in deciding:

- whether a Gaussian is background or foreground
- whether the Gaussian should be kept or discarded

Gaussian Mixture Model

Multiple adaptive Gaussians are necessary to handle acquisition noise, lighting changes, etc.

A linear superposition of Gaussian components provides a richer class of density models than the single Gaussian

Pixel values that do not fit the background distributions (using Mahalanobis distance) are considered as foreground

Motion detection

Motion - Problem definition

Assuming that the scene illumination does not change, the image changes are due to relative motion between the scene objects and the camera (observer).

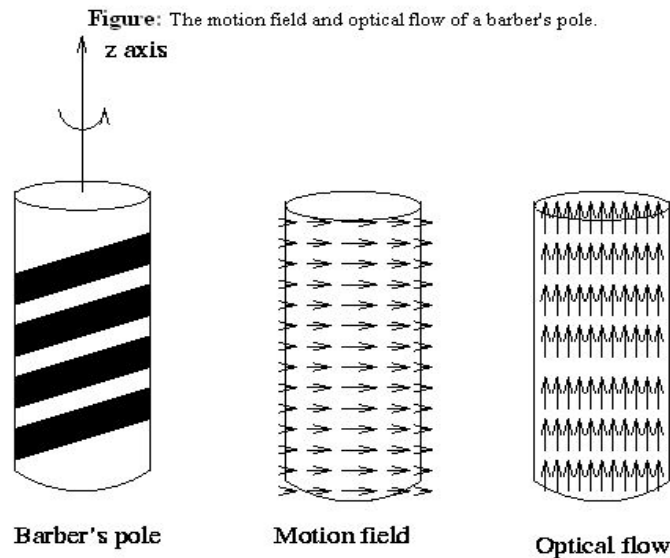
There are three possibilities:

- Stationary camera, moving objects.
- Moving camera, stationary objects.
- Moving camera, moving objects.

Visible Motion and True Motion

Generally, optical flow corresponds to the motion field, but not always.

For example, the motion field and optical flow of a rotating barber's pole are different...



Motion field

Motion field is a 2D representation – projection in the image plane of a (generally) 3D motion of points in the scene (typically on surfaces of objects).

To the each point is assigned a velocity vector corresponding to the motion direction and velocity magnitude.

Motion field

Example:

Figure: The motion field of a moving square.

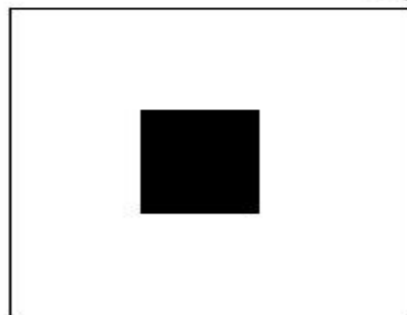


Image 1

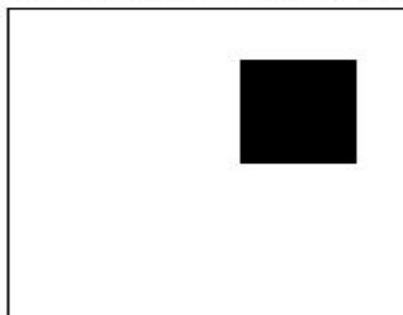
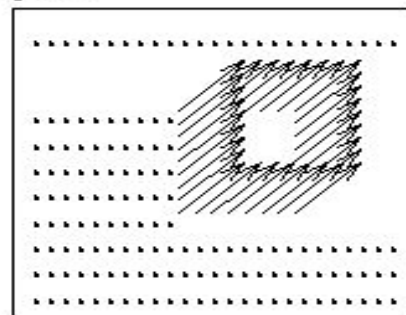


Image 2



Part of motion field

Estimating motion field

1. **Matching of blocks.** Correlation-based matching methods...
2. **Matching of objects.** Interpretation needed. A sparse motion field.
3. **Matching of interest points.**
 - Feature-based methods
 - Problems with repetitive structures.
4. **Optical flow** - a differential technique matching intensities on a pixel-level, exploring spatial and time variations.

Sparse or dense motion field.

Problems for textureless parts of images due to aperture problem.

Matching of blocks

Block matching

Kernel tracking - Template Matching

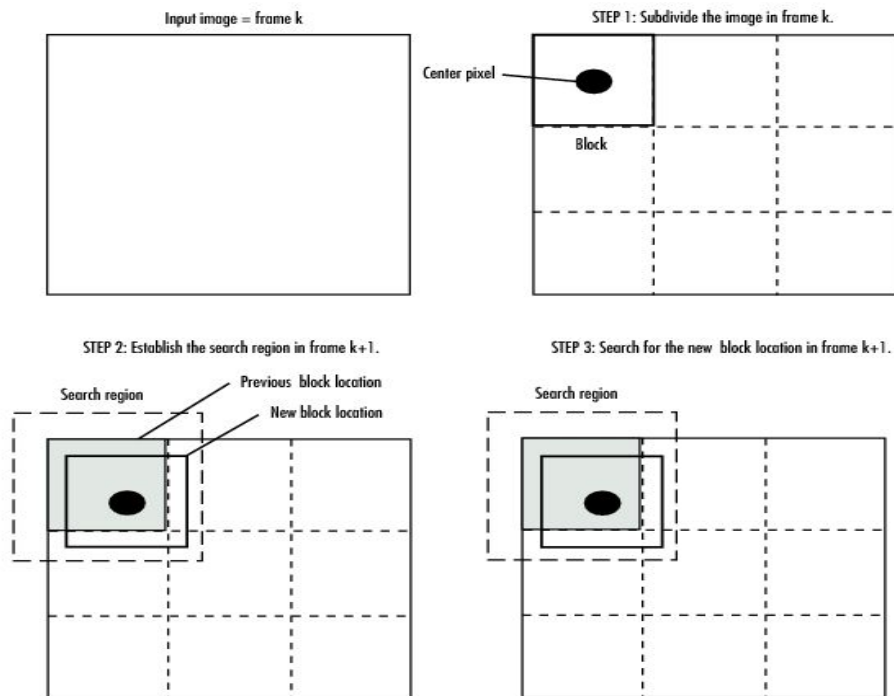
Template Matching: brute force method for tracking single objects

- Define a search area, define a similarity measure
- Place the template defined from the previous frame at each position of the search area and compute a similarity measure between the template and the candidate
- Select the best candidate with the maximal similarity measure

The similarity measure can be a **direct template comparison** or statistical measures **between two probability densities**

Limitation of template matching: high computation costs due to the brute force search □ limit the object search to the vicinity of its previous position

Block matching method



Block matching

Direct comparison:

between template $t(i,j)$ and candidate $g(i,j)$

Histograms comparison:

Bhattacharyya measure (metric) between two distributions (histograms) :

$$d_1(t, g) = \sum_{i=1}^m \sum_{j=1}^n |g(i, j) - t(i, j)|$$

$$d_2(t, g) = \sum_{i=1}^m \sum_{j=1}^n (g(i, j) - t(i, j))^2$$

$$d_3(t, g) = \sum_{i=1}^m \sum_{j=1}^n g(i, j) \cdot t(i, j)$$

$$bd(H_1, H_2) = \sqrt{1 - bc(H_1, H_2)}$$

$$bc(H_1, H_2) = \sum_{i=1}^n \sqrt{H_1(i) \cdot H_2(i)}$$

Silhouette tracking

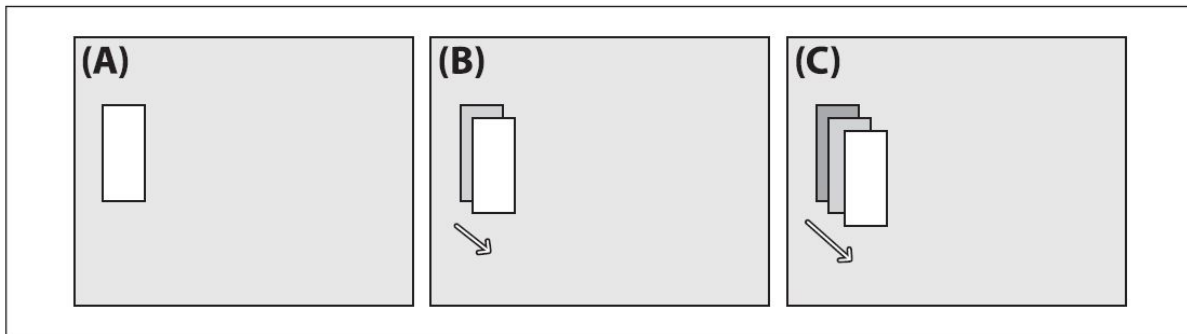
Motion Templates

Motion History Images

Gradient method

Applicable to gesture recognition.

Using motion templates requires a silhouette (or part of a silhouette) of an object.



Normal Flow by Motion Templates

Object silhouette

Motion history images

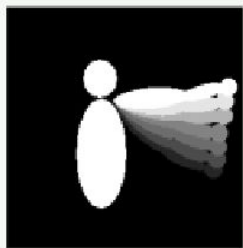
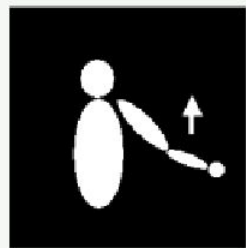
Motion history gradients

Motion segmentation algorithm

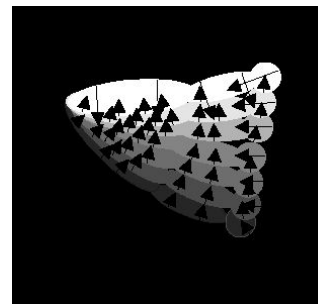
Bradski Davis, Int. Jour. of Mach.
Vision Applications 2001

Binary
silhouette

Motion history
weighted intensity

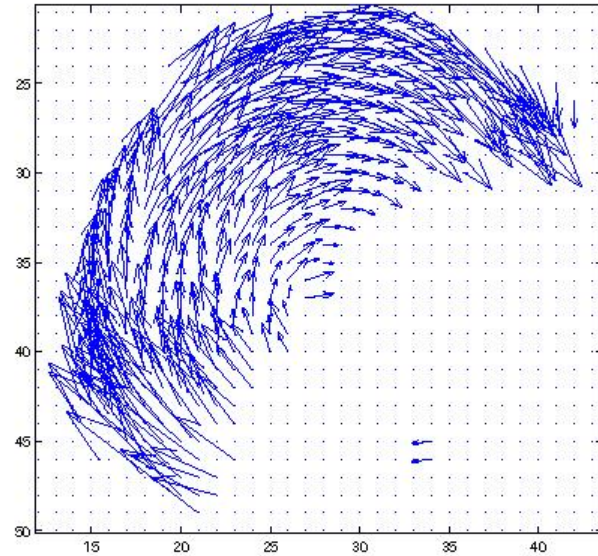
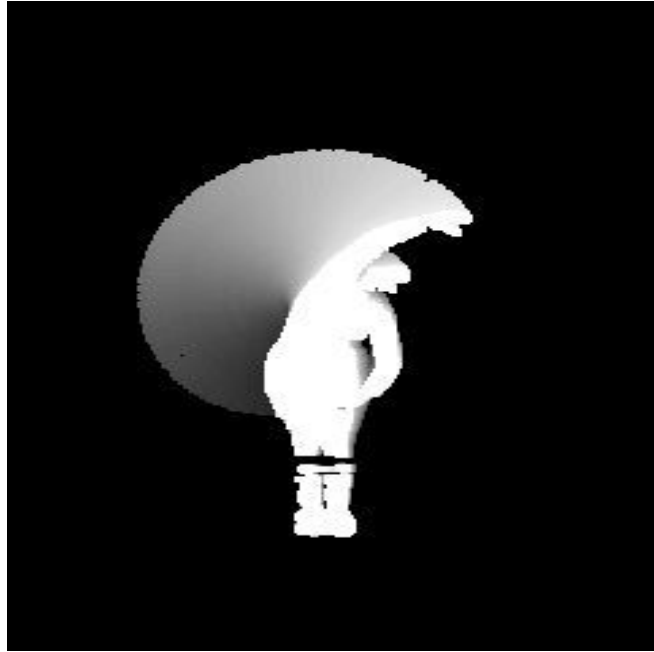


Motion history
gradients



Motion
vectors

Motion History Images



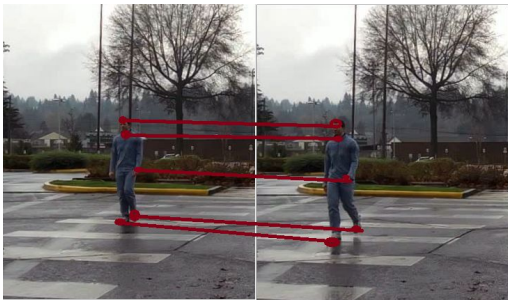
Matching of interest points

- Key-points

Matching of interest points for motion detection

Finding and linking distinctive local features across a sequence of frames to determine how objects are moving.

- detect keypoints and calculate local descriptors (SIFT..)
- match them in the compared frames
- find the matches belonging to the moving object



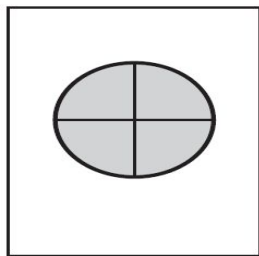
Sparse Optical Flow Lucas-Kanade Tracker

Optical flow

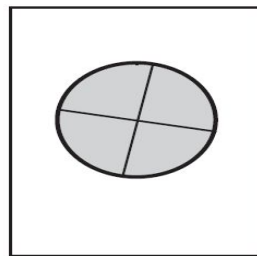
- apparent motion of the same (similar) intensity patterns

Ideally, the optic flow can approximate projections of the three-dimensional motion (i.e., velocity) vectors onto the image plane.

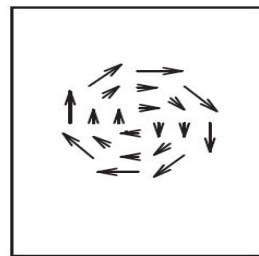
Hence, the optic flow is estimated instead of the motion field



Time t_1

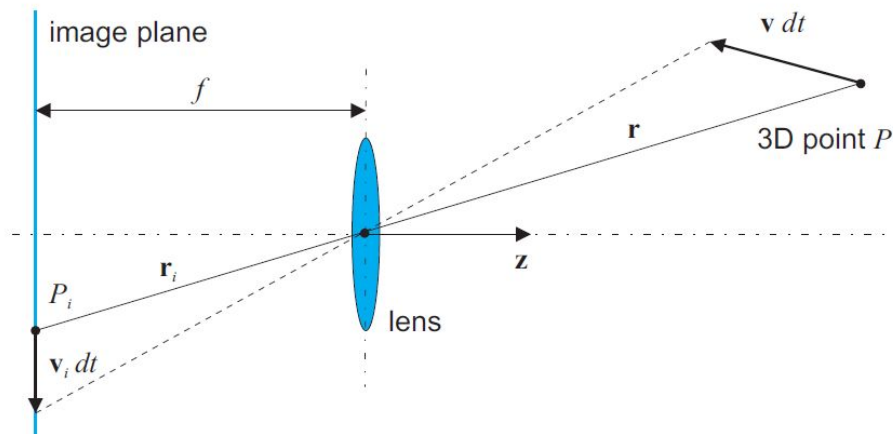


Time t_2



Optic flow

Motionfield / Optical flow



Optical flow is a vector field, from which the motion field can be estimated under certain conditions.

$\mathbf{z}$ – unit vector in the axis z direction.

f – focal length of a lens.

$$v_i = \frac{d}{dt} r_i$$

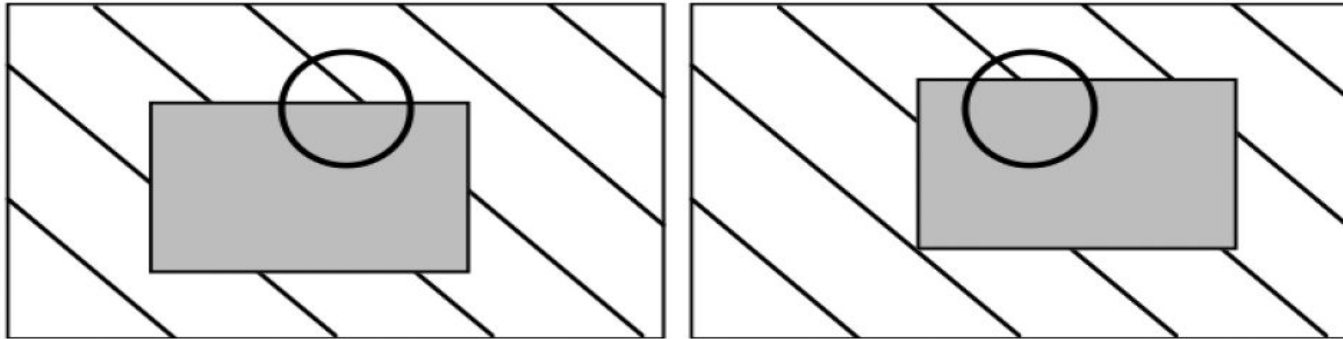
$\frac{d}{dt}$ – velocity in the image plane.

$$\mathbf{v} = \frac{d}{dt} \mathbf{r}$$

$\frac{d}{dt}$ – velocity in the 3D space.

Aperture problem

By analyzing projected images, we always observe a limited area of visible motion only (defined by aperture of the camera or of the area of influence of the used algorithm for motion analysis).

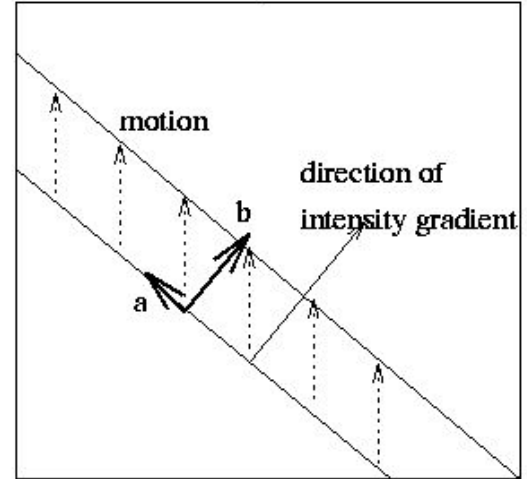


Aperture problem

we are only able to measure the component of optical flow that is in the direction of the intensity gradient.

We are unable to measure the component tangential to the intensity gradient.

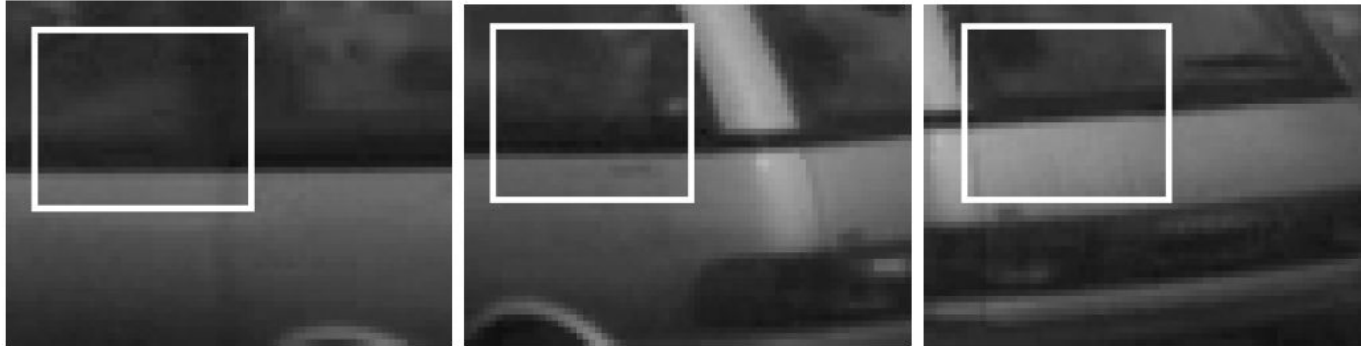
Figure: The aperture problem. We can only measure the component b.



Aperture problem -example

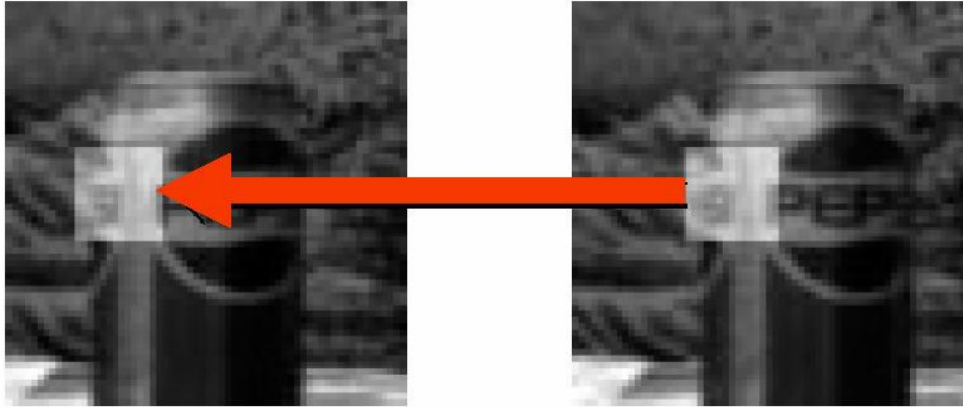
For example, assume that we either only see the inner rectangles, or the whole three images taken at times t , $t + 1$, $t + 2$.

For the inner rectangles, we may conclude an upward translation with a minor rotation....



Optical Flow Assumptions:

1. Brightness constancy

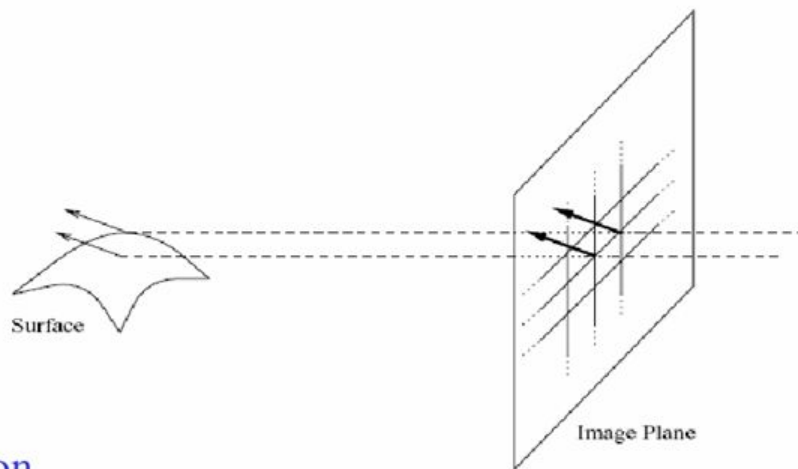


Assumption

Image measurements (e.g. brightness) in a small region remain the same although their location may change.

Optical Flow Assumptions:

2. Spatial Coherence



Assumption

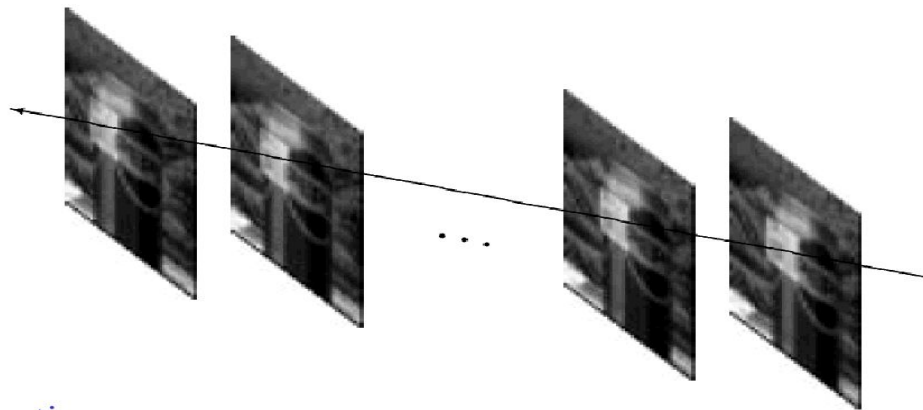
- * Neighboring points in the scene typically belong to the same surface and hence typically have similar motions.
- * Since they also project to nearby points in the image, we expect spatial coherence in image flow.

Optical Flow Assumptions:

3. Temporal persistence

Temporal persistence or “small movements”.

The image motion of a surface patch changes slowly in time. In practice, this means the temporal increments are fast enough relative to the scale of motion in the image that the object does not move much from frame to frame.



Assumption:

The image motion of a surface patch changes gradually over time.

Lucas-Kanade method

Optical Flow: 1D Case

Brightness Constancy Assumption:

$$f(t) \equiv I(\underbrace{x(t), t}_{\text{brightness}}) = I(x(t + dt), t + dt)$$

$$\frac{\partial f(x)}{\partial t} = 0 \quad \text{Because no change in brightness with time}$$

$$\left. \frac{\partial I}{\partial x} \right|_t \left(\frac{\partial x}{\partial t} \right) + \left. \frac{\partial I}{\partial t} \right|_{x(t)} = 0$$

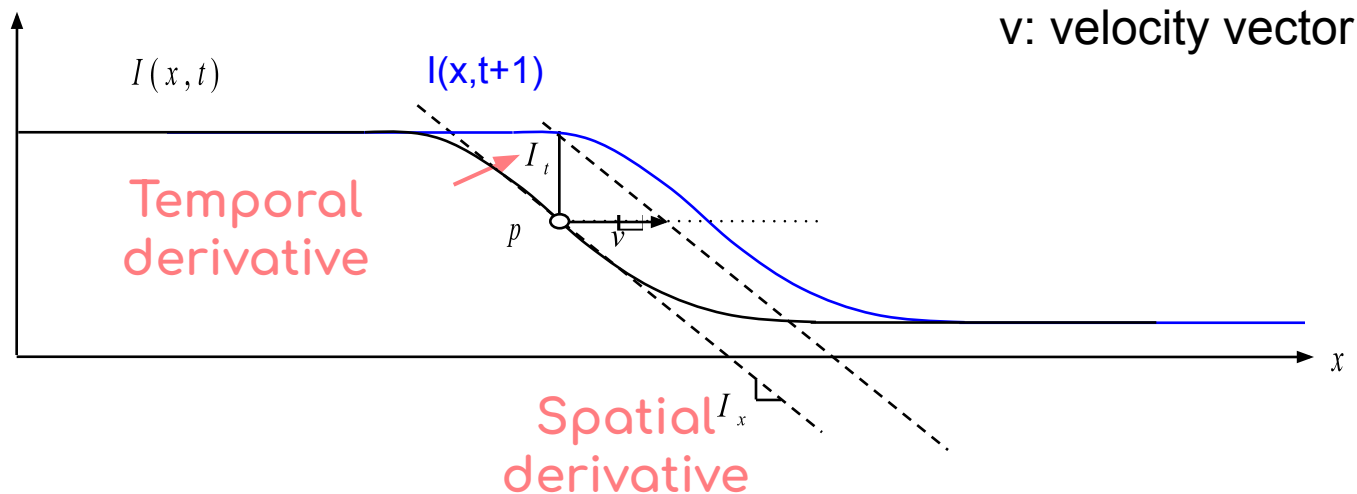
$$I_x \quad v \quad I_t$$

$$\Rightarrow v = \frac{I_t}{I_x}$$

Lucas-Kanade method

Optical Flow: 1D Case

Tracking in the 1D case:



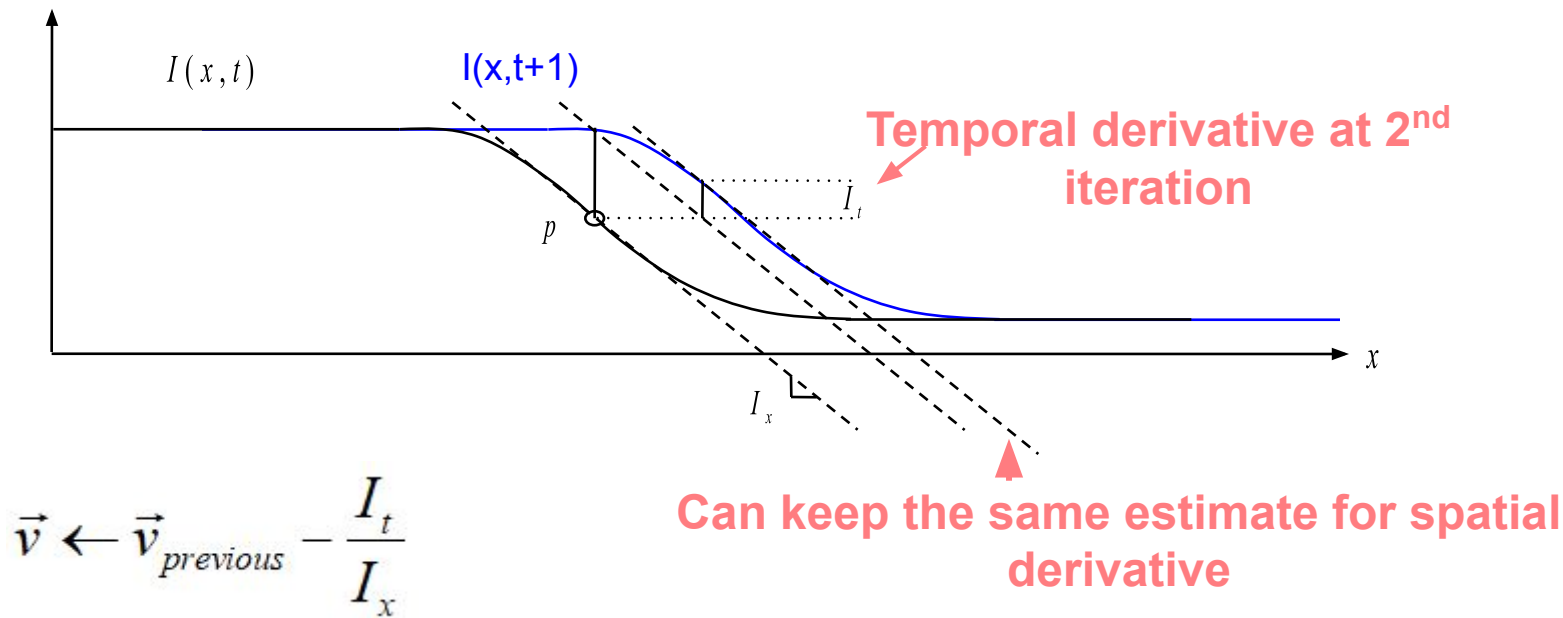
$$I_x = \left. \frac{\partial I}{\partial x} \right|_t \quad I_t = \left. \frac{\partial I}{\partial t} \right|_{x=p} \quad \longrightarrow \quad \vec{v} \approx -\frac{I_t}{I_x}$$

Assumption:
Brightness constancy
Small motion

Lucas-Kanade method

Tracking in the 1D case

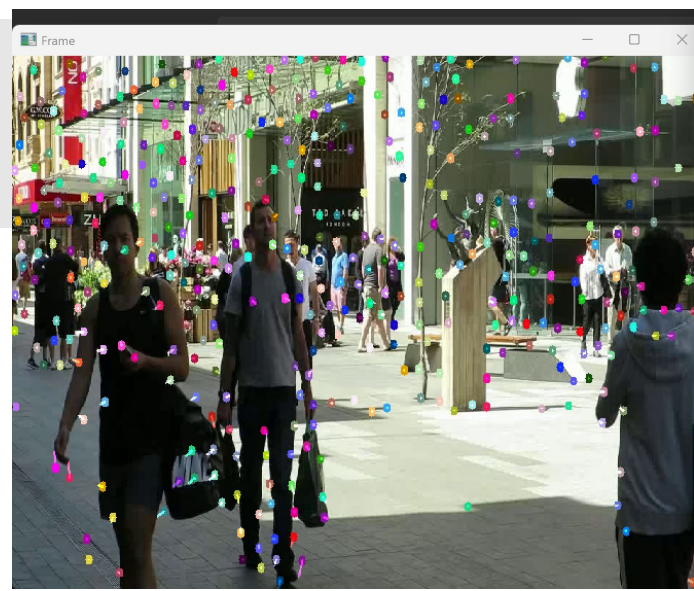
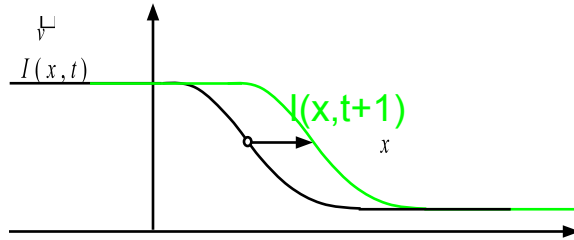
Iterating helps refining the velocity vector



Converges in about 5 iterations

Lucas-Kanade method

From 1D to 2D tracking



To 2D

$$\vec{v} \approx -\frac{I_t}{I_x}$$



$$\left\{ \begin{array}{l} \vec{v} \approx -G^{-1}b \\ G = \sum_{\text{window around } p} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \\ b = \sum_{\text{window around } p} \begin{bmatrix} I_x I_t \\ I_y I_t \end{bmatrix} \end{array} \right.$$

Lucas-Kanade Tracking (KLT) method

KLT in Pyramids

Problem:

we want a large window to catch large motions, but a large window too often breaks the coherent motion assumption!

To circumvent this problem, we can track first over larger spatial scales using an image pyramid and then refine the initial motion velocity assumptions by working our way down the levels of the image pyramid until we arrive at the raw image pixels.

Pyramid processing

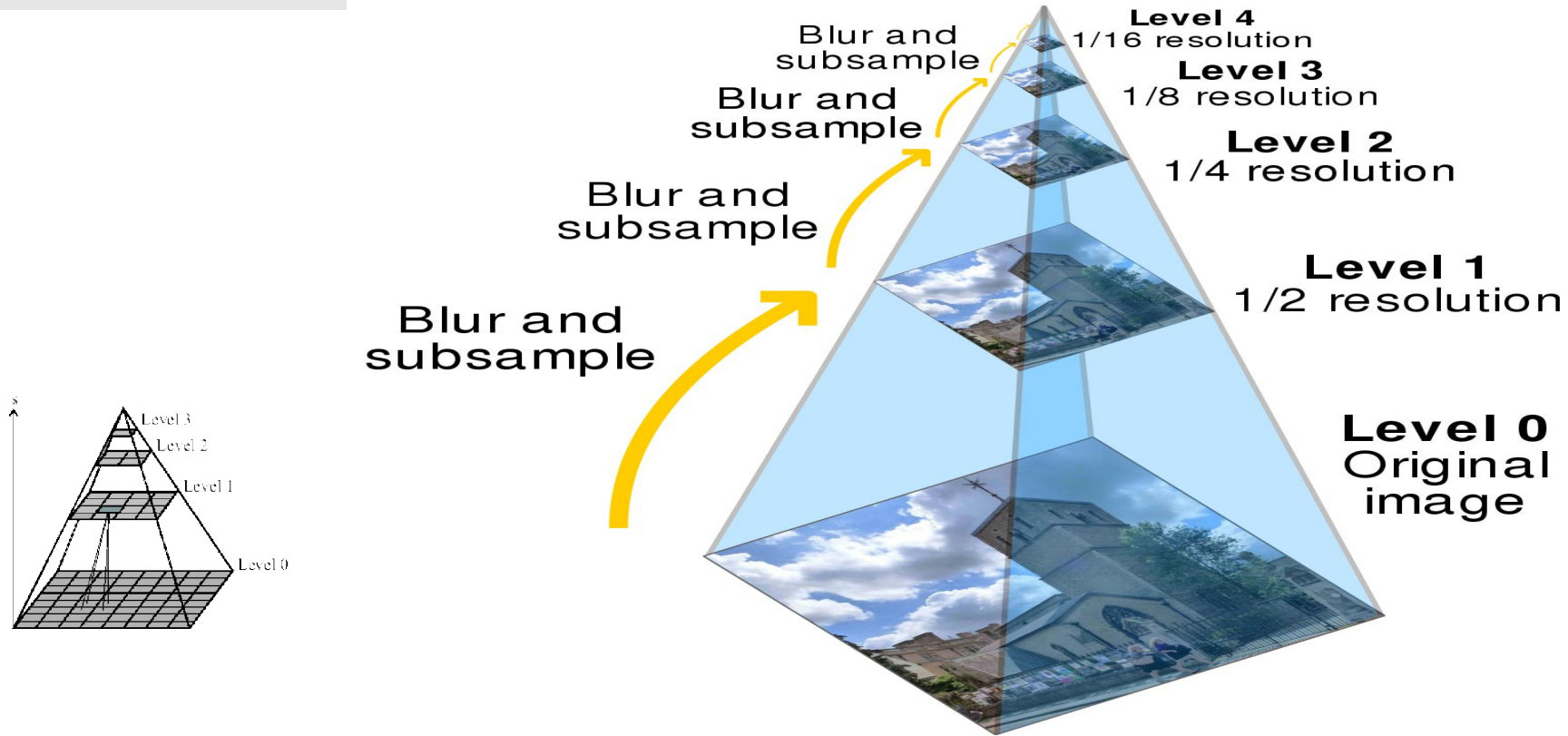
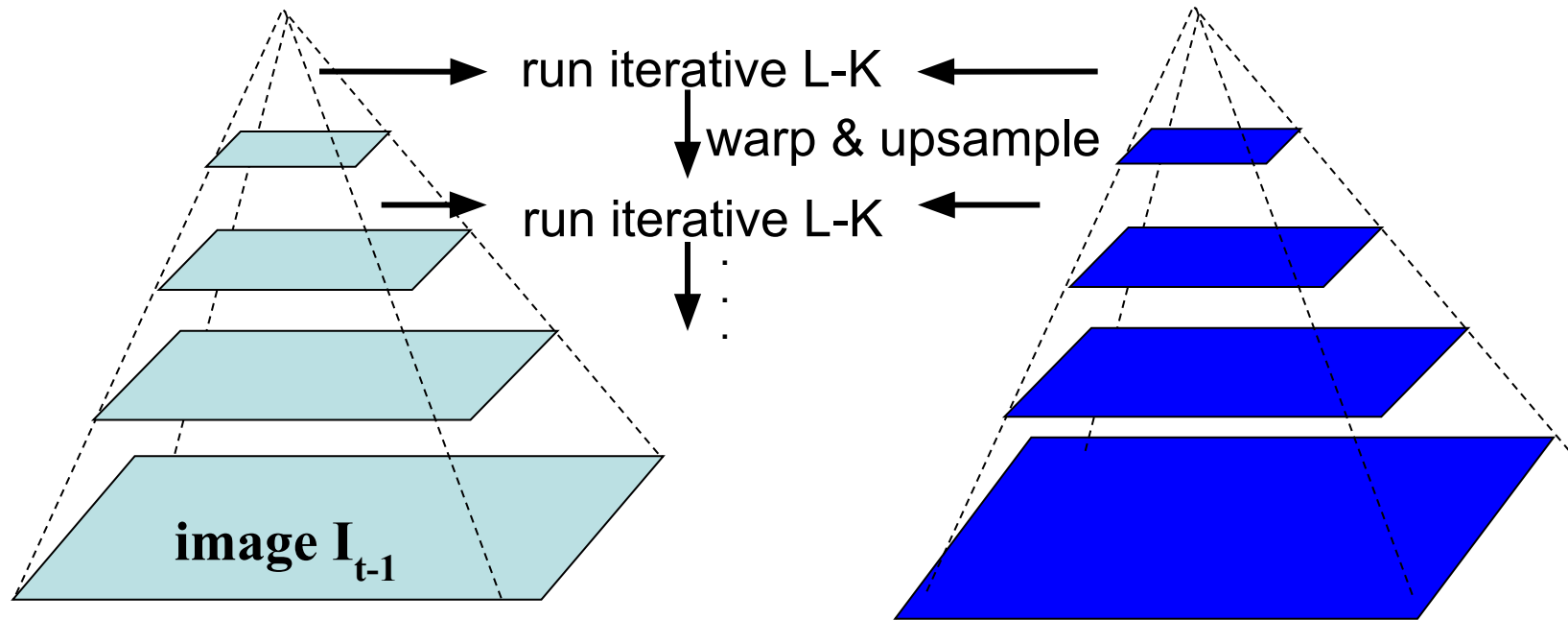


image I

KLT in Pyramids

Coarse-to-fine optical flow estimation



Gaussian pyramid of image I_{t-1}

Gaussian pyramid of image I

Lucas-Kanade Tracking (KLT) method

KLT in Pyramids

First solve optical flow at the top layer and then to use the resulting motion estimates as the starting point for the next layer down.

We continue going down the pyramid in this manner until we reach the lowest level.

Thus we minimize the violations of our motion assumptions and so can track faster and longer motions.

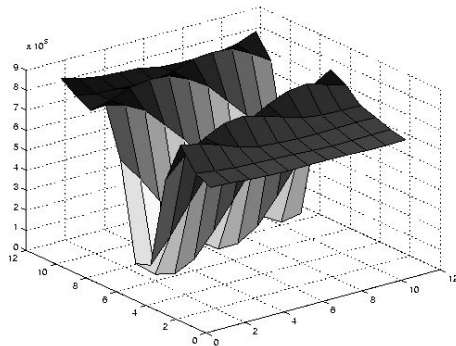
Key points for tracking

local key point detection -see previous lecture



$$\sum \nabla I (\nabla I)^T$$

- large gradients, all the same
- large λ_1 , small λ_2



Key point detection – recap.

Shi and Tomasi: Good Features to Track

Shi and Tomasi:
 $\min \text{Eigenvalue} > Th$

Good corner results as long as the smaller of the **two eigenvalues of the autocorrelation matrix of the second derivative images over a small window** around each point is greater than a minimum threshold.

Shi and Tomasi's method was not only sufficient but in many cases gave more satisfactory results than Harris's method.

Dense Optical Flow

Horn-Schunck method

Horn-Schunck Algorithm

Dense Optical Flow

- An iterative implementation computes the optical flow of **each pixel**.
- computationally time-consuming

Horn-Schunck Algorithm

Smoothness Constraint

Smoothness Constraint

A method for finding the optical flow pattern which assumes that the apparent velocity of the brightness pattern varies smoothly almost everywhere in the image.

Horn-Schunck Algorithm

Minimization Constraint

Minimization Constraint

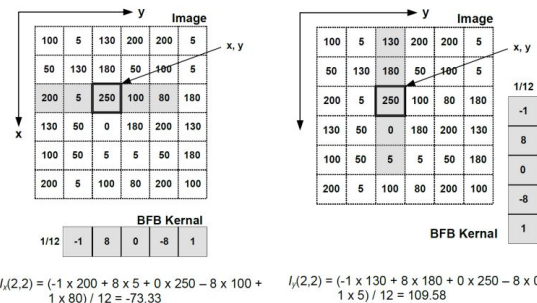
The additional constraint is to minimize the sum of the errors in optical flow

...iterative equations are solved to minimize the sum of the errors for the rate of change of image gradient intensity and obtain the velocity.

Horn and Schunck Algorithm

To solve u and v using the Horn-Schunck method:

- Compute I_x and I_y by using the Sobel convolution kernel, $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$, and its transposed form for each pixel in the first image.
- Compute I_t between images 1 and 2 using the $\begin{bmatrix} -1 & 1 \end{bmatrix}$ kernel.
- Assume the previous velocity to be 0, and compute the average velocity for each pixel using $\begin{bmatrix} 0 & 1 & 0; \\ 1 & 0 & 1; \\ 0 & 1 & 0 \end{bmatrix}$ as a convolution kernel.
- Iteratively solve for u and v .



$$I_x(2,2) = (-1 \times 200 + 8 \times 5 + 0 \times 250 - 8 \times 100 + 1 \times 80) / 12 = -73.33$$

$$I_y(2,2) = (-1 \times 130 + 8 \times 180 + 0 \times 250 - 8 \times 0 + 1 \times 5) / 12 = 109.58$$

$$I(x,y,t) = I(x+u, y+v, t+1),$$

$$\varepsilon = u I_x + v I_y + I_t = 0,$$

u and v are the horizontal and vertical motion vectors of optical flow

see : Tutorial of Motion Estimation Based on Horn-Schunck Optical Flow Algorithm in MATLAB®
Darun Kesrarat¹ and Vorapoj Patana

Advantages / Disadvantages of the Horn-Schunck algorithm

Advantages :

it yields a **high density of flow vectors**, i.e. the flow information missing in inner parts of homogeneous objects is filled in from the motion boundaries.

Disadvantages:

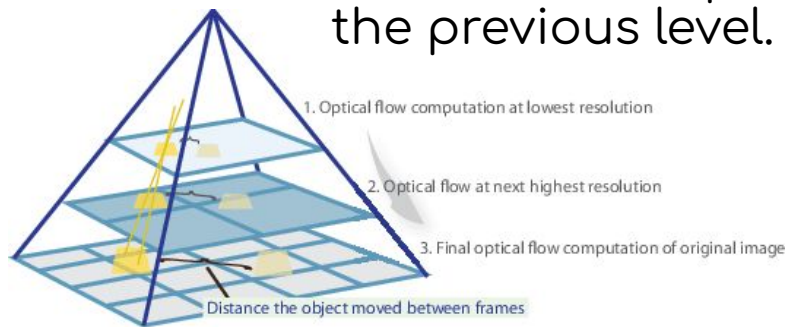
- it is more **sensitive to noise** than local methods.
- The Horn-Schunck method is very unstable in case of illumination artefacts in image sequences (due to the assumed brightness constancy).

Gunnar Farneback Optical Flow

Dense Optical Flow

- Model of image intensity using quadratic function
- Model the displacement by quadratic function
- Minimize the sum of differences between the models and the image

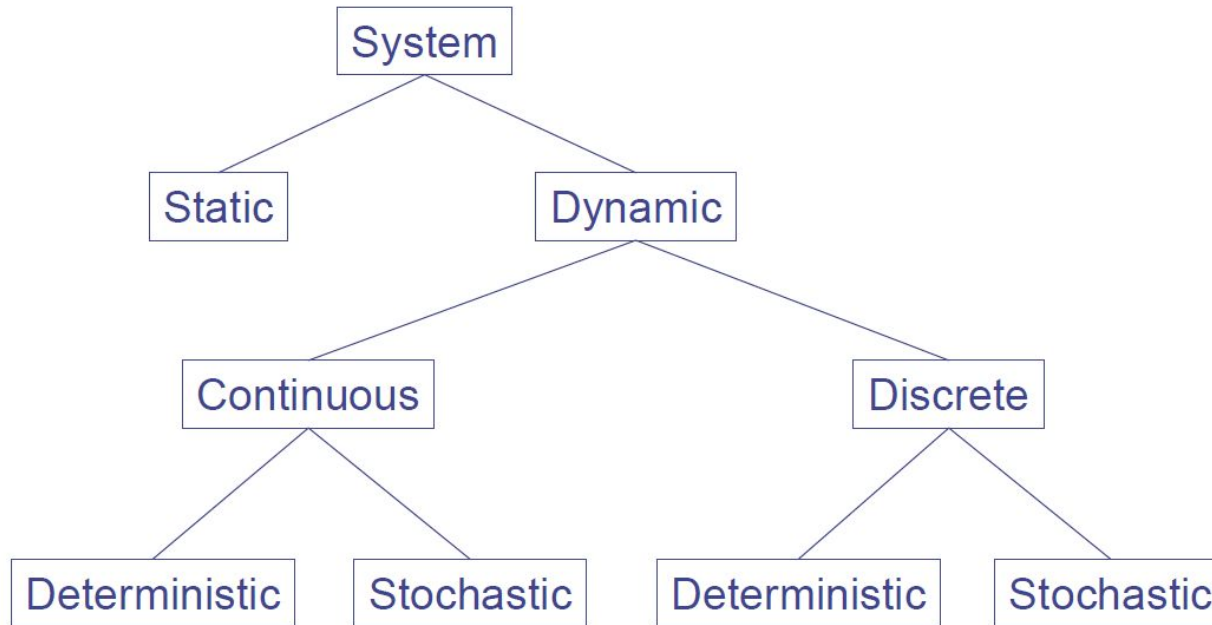
The Farneback algorithm generates an image pyramid, where each level has a lower resolution compared to the previous level.



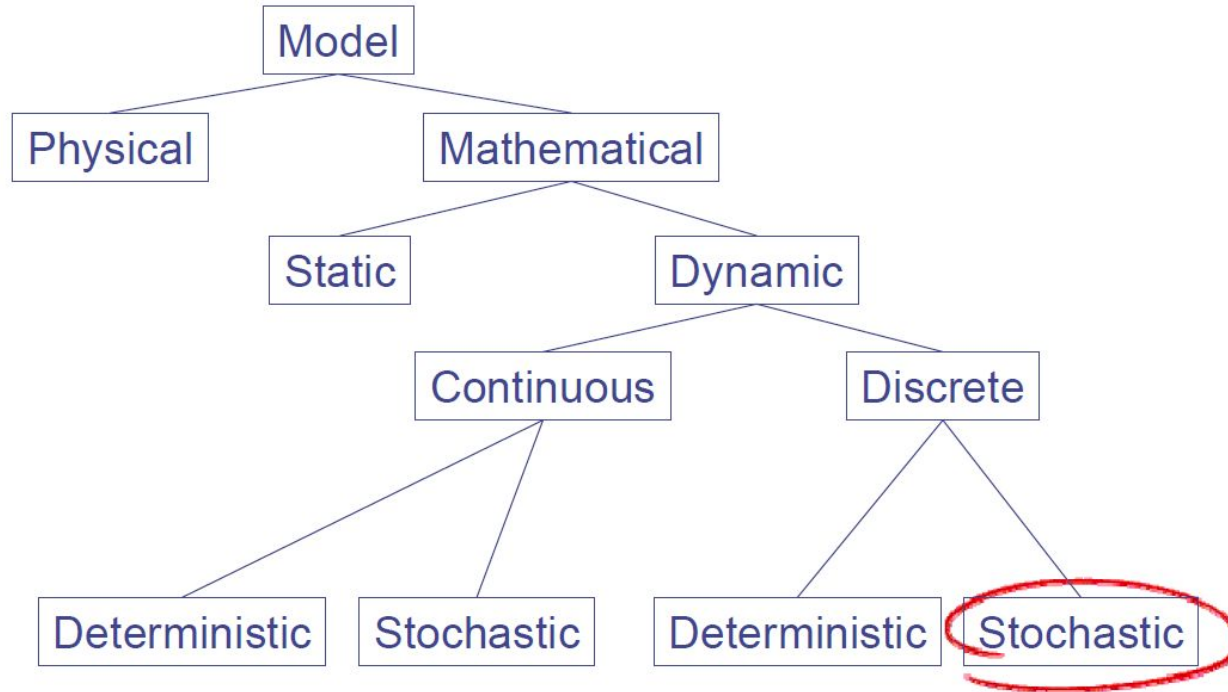
Dynamic systems

Estimation - Prediction

System



Model of system



Estimation - Prediction

In a long image sequence, if the dynamics of the moving object is known, prediction can be made about the positions of the objects in the current image.

This information can be combined with the actual image observation to achieve more robust results.

Tracking formulated using the Hidden Markov model (HMM)

Hidden Markov model (HMM):

x_k is called the hidden state and z_k is called the observation
Markov chain

$$p(x_k | x_1, \dots, x_{k-1}) = p(x_k | x_{k-1})$$

As the result

$$p(z_k | x_1, \dots, x_k) = p(z_k | x_k)$$

$$p(x_1, \dots, x_k, z_1, \dots, z_k) = p(x_1) p(z_1 | x_1) \prod_{i=2}^k [p(x_i | x_{i-1}) p(z_i | x_i)]$$



Tracking formulated using the Hidden Markov model

The state x_k usually consists of the position, the velocity, the acceleration, and other features of the object

The observation z_k is usually the video frame at current time instance.

The transition probability is modelled using dynamic model such as constant velocity model, or constant acceleration model

Example: A constant velocity dynamic model

Kalman filter

Kalman filter

The Kalman filter is a set of mathematical equations that provides an efficient computational (recursive) means to **estimate the state of a process, in a way that minimizes the mean of the squared error.**

History:

Kálmán's ideas on filtering were initially met with vast skepticism, so much so that he was forced to do the first publication of his results in mechanical engineering, rather than in electrical engineering or systems engineering.

Kálmán had more success in presenting his ideas, however, while visiting Stanley F. Schmidt at the NASA Ames Research Center in 1960. This led to the use of Kálmán filters during the Apollo program, and furthermore, in the NASA Space Shuttle...

Kalman filter equations allow us to recursively calculate $\mathbf{x}_t$ by combining: prior knowledge, predictions from dynamic systems models, and noisy measurements.

Example: At every measurement epoch we wish to know the best possible estimate of the location of the train (or more precisely, the location of the radio antenna mounted on the train roof). Information is available from two sources:

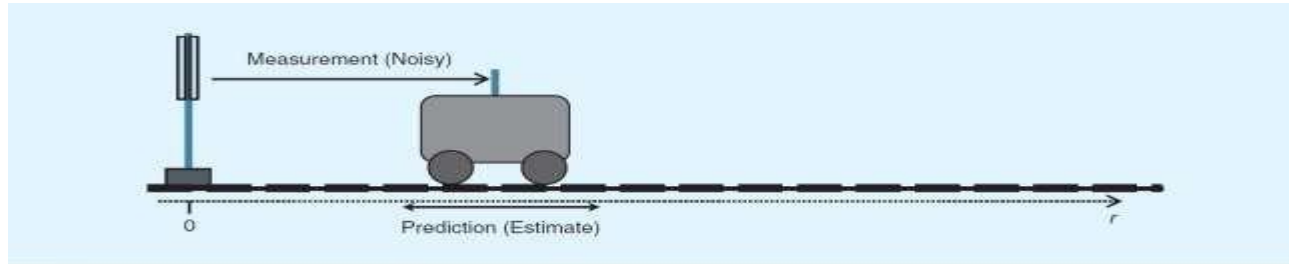
1. predictions based on the last known position and velocity of the train and
2. measurements from a radio ranging system deployed at the track side.

The information from the predictions and measurements are combined to provide the **best possible estimate** of the location of the train.

Kalman filter – example

- simple one-dimensional tracking problem, particularly that of a train moving along a railway line (see Figure 1).

The state vector $\mathbf{x}_t$ contains the position and velocity of the train $\mathbf{x}_t = \begin{bmatrix} x_t \\ \dot{x}_t \end{bmatrix}$.



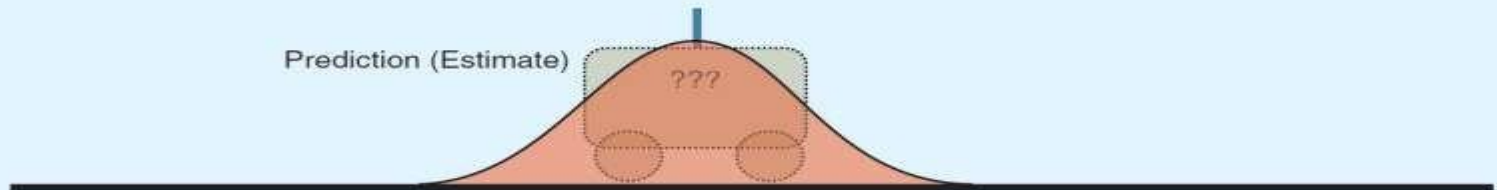
[FIG1] This figure shows the one-dimensional system under consideration.

The train driver may apply a braking or accelerating input to the system, which we will consider here as a function of an applied force f_t and the mass of the train m . Such control information is stored within the control vector $\mathbf{u}_t$

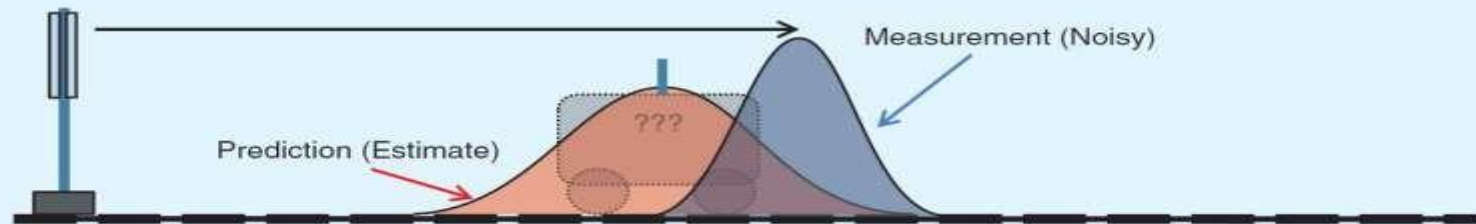
$$\mathbf{u}_t = \frac{f_t}{m}.$$



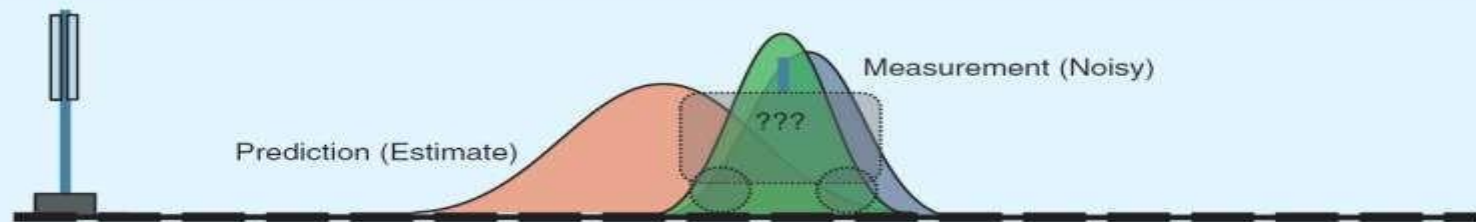
[FIG2] The initial knowledge of the system at time $t = 0$. The red Gaussian distribution represents the pdf providing the initial confidence in the estimate of the position of the train. The arrow pointing to the right represents the known initial velocity of the train.



[FIG3] Here, the prediction of the location of the train at time $t = 1$ and the level of uncertainty in that prediction is shown. The confidence in the knowledge of the position of the train has decreased, as we are not certain if the train has undergone any accelerations or decelerations in the intervening period from $t = 0$ to $t = 1$.



[FIG4] Shows the measurement of the location of the train at time $t = 1$ and the level of uncertainty in that noisy measurement, represented by the blue Gaussian pdf. The combined knowledge of this system is provided by multiplying these two pdfs together.



[FIG5] Shows the new pdf (green) generated by multiplying the pdfs associated with the prediction and measurement of the train's location at time $t = 1$. This new pdf provides the best estimate of the location of the train, by fusing the data from the prediction and the measurement.

Kalman filter

The Kalman filter is an efficient recursive filter that estimates the state of a dynamic system from a series of incomplete and noisy measurements.

→ Optimal estimator

Used to estimate system states that can only be observed indirectly or inaccurately by the system itself

→ Recursive calculation!

Kalman filter

Kalman filter is a set of mathematical equations that provides an efficient computational (recursive) means to estimate the state of a process, in the way that **minimizes the mean of squared error**.

Kalman filter is the estimator that satisfies two criteria:

- 1) the mean value of the estimate state should be equal to the mean value of the true state
- 1) we want to find the estimator with the smallest possible error variance

Kalman Filter - calculation

- 1.step : dynamic system model definition - State equation and Output equation
2. step : start recursive computation
3. - n steps : iterate, iterate, iterate.....

Model of a dynamic linear system - used in Kalman filter

Dynamic linear system is described by State equation (1) and Output equation (2).

$\mathbf{x}_t$ - State in the time t :

$$(1) \quad \mathbf{x}_t = \underbrace{\mathbf{A} * \mathbf{x}_{t-1}}_{\text{Dynamic term}} + \underbrace{\mathbf{B} * \mathbf{u}_t}_{\text{Control term}} + \underbrace{\mathbf{w}_t}_{\text{Noise term}}$$

$\mathbf{A}$ - state transition matrix $\mathbf{B}$ - matrix $\mathbf{w}_t$ - process noise
 $\mathbf{x}_{t-1}$ - state in the time $t-1$ $\mathbf{u}_t$ - control signal

$\mathbf{z}_t$ - measured output:

$$\mathbf{z}_t = \underbrace{\mathbf{H} * \mathbf{x}_t}_{\text{Noise term}} + \underbrace{\mathbf{v}_t}_{\text{Noise term}}$$

$\mathbf{H}$ - measurement matrix $\mathbf{v}_t$ - measurement noise
 $\mathbf{x}_t$ - state in the time t

Dynamic linear system

$$(1) \quad X_t = A * x_{t-1} + B * u_t + w_t$$

The first equation - **State equation** shows that the system state variable is dependent on

- the previous system state,
- the system control inputs and
- a random variable which represents the **process noise** and uncertainty of the model.

$$(2) \quad z_t = H * x_t + v_t$$

The second equation - **Output equation** shows that the measured system output is dependent on

- the current system state and
- the **measurement noise**.

In special case, the measured system output can be equal to the state variable, but in general are different.

Dynamic linear system

The first equation includes:

- $\mathbf{x}_t$ is the **state vector** containing the terms of interest for the system (e.g., position, velocity, heading...) at time t
- $\mathbf{u}_t$ is the vector containing any **control inputs** (steering angle, throttle setting, braking force)
- $\mathbf{A}_t$ is the **state transition matrix** which applies the effect of each system state parameter at time $t-1$ on the system state at time t (e.g., the position and velocity at time $t-1$ both affect the position at time t)
- $\mathbf{B}_t$ is the **control input matrix** which applies the effect of each control input parameter in the vector $\mathbf{u}_t$ on the state vector (e.g., applies the effect of the throttle setting on the system velocity and position)
- $\mathbf{w}_t$ is the vector containing the **process noise** terms for each parameter in the state vector. The process noise is assumed to be drawn from a zero mean multivariate normal distribution with covariance given by the covariance matrix $\mathbf{Q}_t$.

Dynamic linear system

The second equation includes:

- $\mathbf{z}_t$ is the **vector of measurements**
- $\mathbf{H}_t$ is the **transformation matrix** that maps the state vector parameters into the measurement domain
- $\mathbf{v}_t$ is the vector containing the **measurement noise** terms for each observation in the measurement vector.

Like the process noise, the measurement noise is assumed to be zero mean Gaussian white noise with covariance $\mathbf{R}_t$

Example: We want to model a vehicle going in a straight line.

Let's say that we are able to change the acceleration a and measure the position p every $T=\Delta t$ [seconds].

The state x_t consists of :

vehicle position	$p_t (= x_t)$
and velocity	$v_t (= \dot{x}_t)$

Control information:

applied force	f_t
the mass of the train	m

Example

$$x_t = x_{t-1} + (\dot{x}_{t-1} \times \Delta t) + \frac{f_t(\Delta t)^2}{2m}$$

$$\dot{x}_t = \dot{x}_{t-1} + \frac{f_t \Delta t}{m}.$$

$$\begin{bmatrix} x_t \\ \dot{x}_t \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_{t-1} \\ \dot{x}_{t-1} \end{bmatrix} + \begin{bmatrix} \frac{(\Delta t)^2}{2} \\ \Delta t \end{bmatrix} \frac{f_t}{m}.$$

Víme z fyziky

*Newtonovy pohybové zákony,
zákon síly:*

$$a = F/m$$

Rovnoměrně zrychlený pohyb

$$a = \text{konšt.}$$

$$v = at$$

$$s = \frac{1}{2} at^2$$

Example

Measurement in time $T=\Delta t$

State equation and output equation :

$$x_{k+1} = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} T^2/2 \\ T \end{bmatrix} u_k + w_k$$

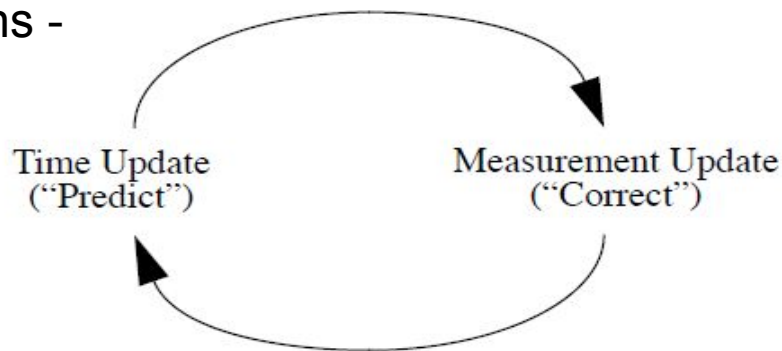
$$y_k = \begin{bmatrix} 1 & 0 \end{bmatrix} x_k + z_k$$

$$\mathbf{A} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \text{ and } \mathbf{B} = \begin{bmatrix} \frac{(\Delta t)^2}{2} \\ \Delta t \end{bmatrix}.$$

We need to estimate the state x

Kalman filter

- the time update equations -
 - predictor equations
- the measurement update equations -
 - corrector equations



Kalman filter

Kalman filter - two groups of the equations :

time update equations

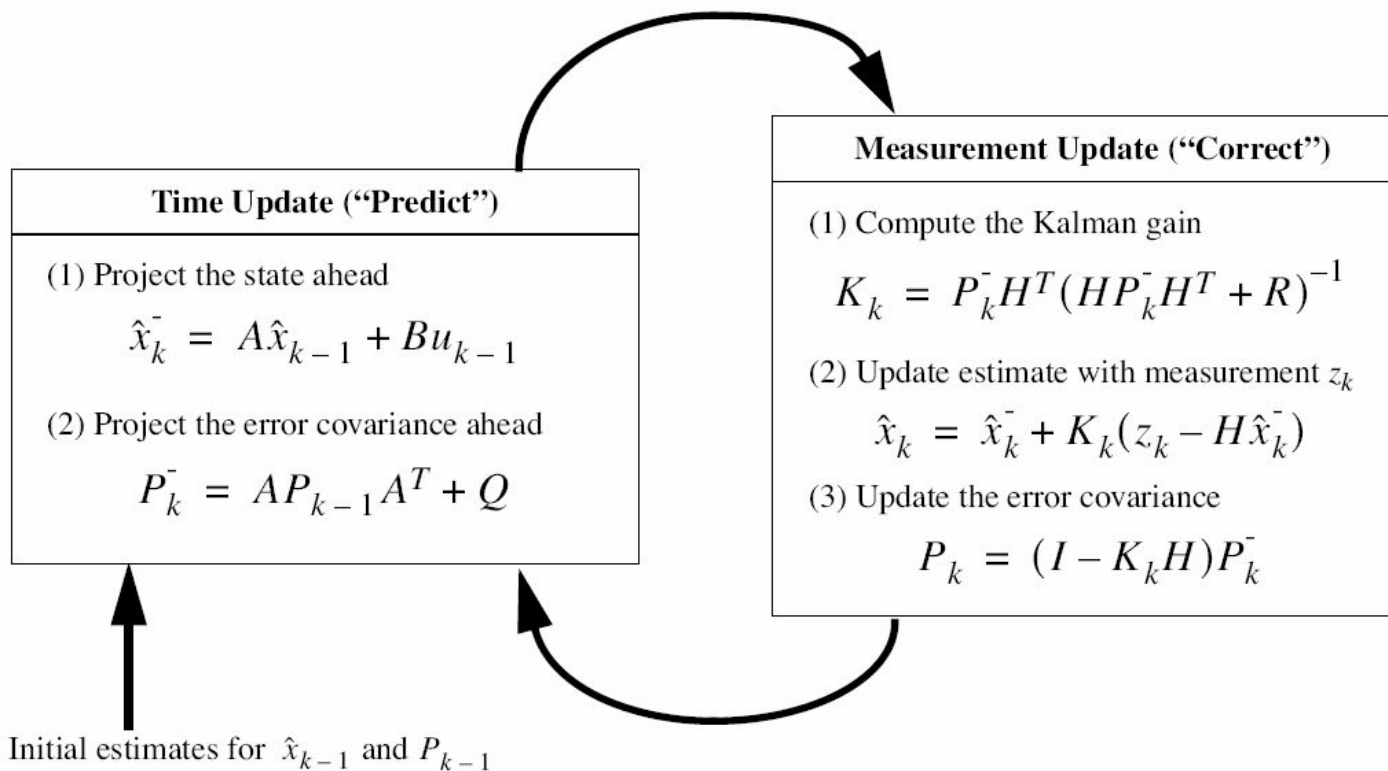
The time update equations are responsible for projecting forward (in time) the current state and error covariance estimates to obtain the *a priori estimates for the next time step*.

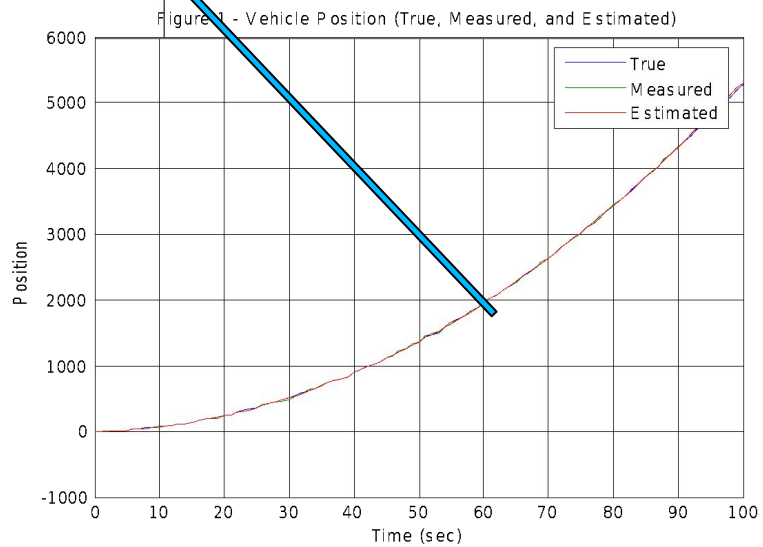
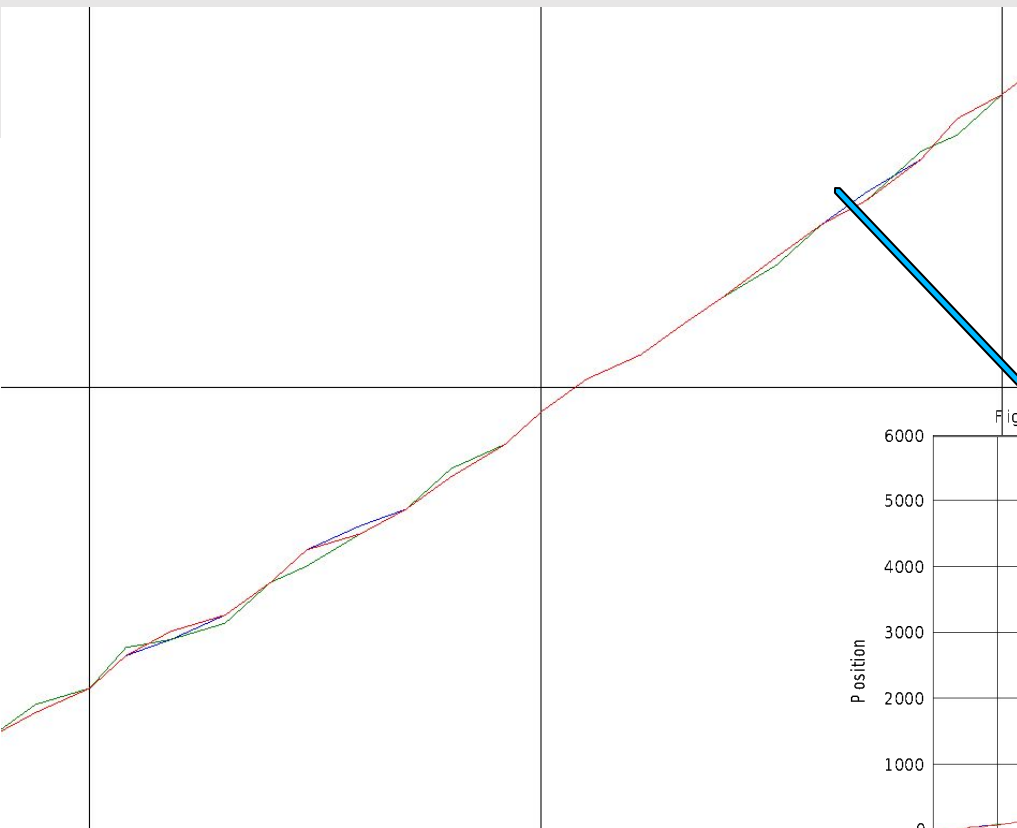
and

measurement update equations.

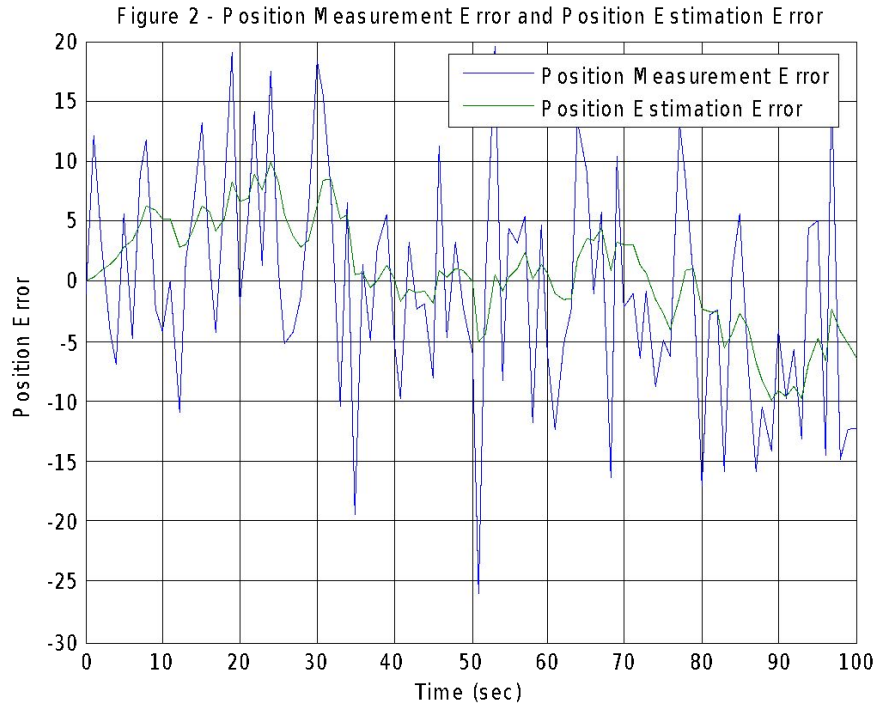
The measurement update equations are responsible for the feedback - for *incorporating of a new measurement into the a priori estimate to obtain an improved a posteriori estimate.*

Kalman filter





Example



Recap. Kalman Filter - Summary

The true state of the system $\mathbf{x}_t$ cannot be directly exact observed,
and the Kalman filter provides an algorithm to determine an estimate $\mathbf{x}_t$ by combining models of the system and noisy measurements of certain parameters or linear functions of parameters.

Extended Kalman filter

- an extension of linear Kalman filter theory to nonlinear systems